A Assets used

Table 2: Assets used in preparing this paper, along with licenses and links

Asset type	Asset name	Link	License	Citation
Code	TransformerLens	GitHub: TransformerLens	MIT	[37]
Code	SAELens	Github: SAELens	MIT	[5]
Data	OpenWebText	HuggingFace: OpenWebText	CC0-1.0	[21]
Model	GPT2-small	HuggingFace: GPT2	MIT	[44]
Model	Pythia-410M	HuggingFace: Pythia-410M	Apache-2.0	[2]
Model	Pythia-1.4B	HuggingFace: Pythia-1.4B	Apache-2.0	[2]

B Compute details

The most compute-intensive parts of the research presented in this work were training the SAEs and transcoders used in Section 4.2, along with the set of GPT2-small transcoders used in Sections 5.1 and 5.2. Training all of these SAEs and transcoders involved GPUs. The SAEs and transcoders from Section 4.2 were trained on an internal cluster using an A100 GPU with 80 GB of VRAM. The VRAM used by each training run ranged from approximately 16 GB for the GPT2-small runs to approximately 60 GB for the Pythia-1.4B runs. The time taken by each training run ranged from approximately 30 minutes for the GPT2-small transcoders/SAEs to approximately 3.5 hours for the Pythia-1.4B runs.

The transcoders that were trained on each layer of GPT2-small were trained using a cloud provider, with a similar amount of time and VRAM used per training run. For these transcoders, a hyperparameter sweep was performed that involved approximately 200 training runs, which did not produce results used in the final paper.

No significant amount of storage was used, as datasets were streamed during training.

In addition to these training runs, our case studies were carried out on internal cluster nodes with GPUs. These case studies used no more than 6 GB of VRAM. The total amount of compute used during each case study is variable (depending on how in-depth one wants to investigate a case study), but is *de minimis* in comparison to the training runs. The same goes for the computation of top activating examples used in Section 4.1.

C SAE details

Sparse autoencoders (SAEs) are autoencoders trained to decompose a model's activations at a given point into a sparse linear combination of feature vectors. As a hypothetical example, given the input "Sally threw the ball to *me*", an SAE might decompose the model's activations on the token `me` into a linear combination of a "personal pronoun" feature vector, an "indirect object" feature, and a "playing sports" feature—where all of these feature vectors are automatically learned by the SAE. An SAE's architecture can be expressed as

$$\mathbf{z_{SAE}}(\mathbf{x}) = \mathrm{ReLU}\left(\mathbf{W_{enc}}\mathbf{x} + \mathbf{b_{enc}}\right) \tag{9}$$

$$\mathrm{SAE}(\mathbf{x}) = \mathbf{W_{dec}}\mathbf{z_{SAE}}(\mathbf{x}) + \mathbf{b_{dec}}, \tag{10}$$

where $\mathbf{W_{enc}} \in \mathbb{R}^{d_{\text{features}} \times d_{\text{model}}}$, $\mathbf{W_{dec}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{features}}}$, $\mathbf{b_{enc}} \in \mathbb{R}^{d_{\text{features}}}$, $\mathbf{b_{dec}} \in \mathbb{R}^{d_{\text{model}}}$, d_{features} is the number of feature vectors in the SAE, and d_{model} is the dimensionality of the model activations. Usually, d_{features} is far greater than d_{model}.

Intuitively, Equation 9 transforms the neuron activations $\mathbf{x}$ into a sparse vector of SAE feature activations $\mathbf{z_{SAE}}(\mathbf{x})$. Each feature in an SAE is associated with an "encoder" vector (the i-th row of $\mathbf{W_{enc}}$) and a "decoder" vector (the i-th column of $\mathbf{W_{dec}}$). Equation 10 then reconstructs the original activations as a linear combination of decoder vectors, weighted by the feature activations.

The basic loss function on which SAEs are trained is

$$\mathcal{L}_{SAE}(\mathbf{x}) = \underbrace{\|\mathbf{x} - \mathrm{SAE}(\mathbf{x})\|_2^2}_{\text{reconstruction loss}} + \underbrace{\lambda_1 \|\mathbf{z_{SAE}}(\mathbf{x})\|_1}_{\text{sparsity penalty}}, \tag{11}$$

where λ_1 is a hyperparameter and $\|\cdot\|_1$ denotes the L_1 norm. The first term in the loss is the *reconstruction loss* associated with the SAE. The second term in the loss is a *sparsity penalty*, which approximately measures the number of features active on each input (the L_1 norm is used as a differentiable approximation of the L_0 "norm"). SAEs are thus pushed to reconstruct inputs accurately with a sparse number of features, with λ_1 controlling the accuracy-sparsity tradeoff. Empirically, the result of this is that SAEs learn to decompose model activations into highly interpretable features [7].

A standard method for quantitatively evaluating an SAE's performance is as follows. To measure its sparsity, evaluate the mean number of features active on any given input (the mean L_0). To measure its accuracy, replace the original language model's activations with the SAE's reconstructed activations and measure the change in the *language model's loss* (in this paper, this is the cross entropy loss for next token prediction).

D Detailed description of circuit analysis

D.1 Notation

$\mathbf{x_{pre}^{(l,t)}}$ denotes the hidden state for token t at layer l before the attention sublayer.

$\mathbf{x_{mid}^{(l,t)}}$ denotes the hidden state for token t at layer l before the MLP sublayer.

When we want to refer to the hidden state of the model for all tokens, we will do by omitting the token index, writing $\mathbf{x_{pre}^{(l,1:t)}}$ and $\mathbf{x_{mid}^{(l,1:t)}}$. These are matrices of size $\mathbb{R}^{d_{\text{model}} \times n_{\text{tokens}}}$, where d_{model} is the dimensionality of model activation vectors and n_{tokens} is the number of input tokens.

The MLP sublayer at layer l is denoted by $\mathrm{MLP}^{(l)}(\cdot)$. Similarly, the transcoder for the layer l MLP is denoted by $\mathrm{TC}^{(l)}(\cdot)$.

As for attention sublayers: following Elhage et al. [15], each attention sublayer can be decomposed into the sum of n_{heads} independently-acting attention heads. Each attention head depends on the hidden states of all tokens in the input, but also distinguishes the token whose hidden state is to be modified by the attention head. Thus, the output of the layer l attention sublayer for token t is denoted $\sum_{\text{head } h} \mathrm{attn}^{(l,h)}\left(\mathbf{x_{pre}^{(l,t)}}; \mathbf{x_{pre}^{(l,1:t)}}\right)$.

Each attention head can further be decomposed as a sum over "source" tokens. In particular, the output of layer l attention head h for token t can be written as

$$\mathrm{attn}^{(l,h)}\left(\mathbf{x_{pre}^{(l,t)}}; \mathbf{x_{pre}^{(l,1:t)}}\right) = \sum_{\text{source token } s} \mathrm{score}^{(l,h)}\left(\mathbf{x_{pre}^{(l,t)}}, \mathbf{x_{pre}^{(l,s)}}\right) \mathbf{W_{OV}^{(l,h)}} \mathbf{x_{pre}^{(l,s)}} \tag{12}$$

Here, $\mathrm{score}^{(l,h)} : \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}} \to \mathbb{R}$ is a scalar "scoring" function that weights the importance of each source token to the destination token. Additionally, $\mathbf{W_{OV}^{(l,h)}}$ is a low-rank $\mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ matrix that transforms the hidden state of each source token. $\mathrm{score}^{(l,h)}$ is often referred to as the "QK circuit" of attention and $\mathbf{W_{OV}^{(l,h)}}$ is often referred to as the "OV circuit" of attention.

D.2 Derivation of Equation 7

We want to understand what causes feature i' in the transcoder at layer l' to activate on token t. The activation of this feature is given by

$$\mathrm{ReLU}\left(\mathbf{f_{enc}^{(l',i')}} \cdot \mathbf{x_{mid}^{(l',t)}} + b_{enc}^{(l',i')}\right), \tag{13}$$

where $\mathbf{f_{enc}^{(l',i')}}$ is the i'-th row of $\mathbf{W_{enc}}$ for the layer l' transcoder and $b_{enc}^{(l',i')}$ is the learned encoder bias for feature i' in the layer l' transcoder. Therefore, if we ignore the constant bias term $b_{enc}^{(l',i')}$, then, assuming that this feature is active (which allows us to ignore the ReLU), the activation of feature i' depends solely on $\mathbf{f_{enc}^{(l',i')}} \cdot \mathbf{x_{mid}^{(l',t)}}$. Because of residual connections in the transformer, $\mathbf{x_{mid}^{(l',t)}}$ can be decomposed as the sum of the outputs of all previous components in the model. For instance, in a

two-layer model, if $\mathbf{x}_{\mathbf{mid}}^{(\mathbf{2},\mathbf{t})}$ is the hidden state of the model right before the second MLP sublayer, then

$$\mathbf{x}_{\mathbf{mid}}^{(\mathbf{2},\mathbf{t})} = \sum_h \mathrm{attn}^{(2,h)}\left(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{2},\mathbf{t})}; \mathbf{x}_{\mathbf{pre}}^{(\mathbf{2},\mathbf{1}:\mathbf{t})}\right) + \mathrm{MLP}^{(1)}\left(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{1},\mathbf{t})}\right) + \sum_h \mathrm{attn}^{(1,h)}\left(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{1},\mathbf{t})}; \mathbf{x}_{\mathbf{pre}}^{(\mathbf{1},\mathbf{1}:\mathbf{t})}\right). \tag{14}$$

Because of linearity, this means that the amount that $\mathrm{MLP}^{(1)}\left(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{1},\mathbf{t})}\right)$ contributes to $\mathbf{f}_{\mathbf{enc}}^{(\mathbf{2},\mathbf{i}')} \cdot \mathbf{x}_{\mathbf{mid}}^{(\mathbf{2},\mathbf{t})}$ is given by

$$\mathbf{f}_{\mathbf{enc}}^{(\mathbf{2},\mathbf{i}')} \cdot \mathrm{MLP}^{(1)}\left(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{1},\mathbf{t})}\right). \tag{15}$$

This is generally true for understanding the contribution of MLP l to the activation of feature i' in transcoder l', whenever $l < l'$.

Now, if the layer l transcoder is a sufficiently good approximation to the layer l MLP, we can replace the latter with the former: $\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')} \cdot \mathrm{MLP}^{(\mathrm{l})}\left(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})}\right) \approx \mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')} \cdot \mathrm{TC}^{(\mathrm{l})}\left(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})}\right)$. We can further decompose this into individual transcoder features: $\mathrm{TC}^{(\mathrm{l})}\left(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})}\right) = \sum_{\text{feature } j} z_{TC}^{(l,j)}(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})})\mathbf{f}_{\mathbf{dec}}^{(\mathbf{l},\mathbf{j})}$. Thus, again taking advantage of linearity, we have

$$\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')} \cdot \mathrm{MLP}^{(\mathrm{l})}\left(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})}\right) \approx \mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')} \cdot \sum_{\text{feature } j} z_{TC}^{(l,j)}(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})})\mathbf{f}_{\mathbf{dec}}^{(\mathbf{l},\mathbf{j})} \tag{16}$$

$$= \sum_{\text{feature } j} z_{TC}^{(l,j)}(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})})\left(\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')} \cdot \mathbf{f}_{\mathbf{dec}}^{(\mathbf{l},\mathbf{j})}\right) \tag{17}$$

Therefore, the attribution of feature i in transcoder l on token t is given by

$$z_{TC}^{(l,j)}(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})})\left(\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')} \cdot \mathbf{f}_{\mathbf{dec}}^{(\mathbf{l},\mathbf{j})}\right). \tag{18}$$

D.3 Attribution through attention heads

So far, we have addressed how to find the attribution of a lower-layer transcoder feature directly on a higher-layer transcoder feature at the same token. But transcoder features can also be mediated by attention heads. We will thus extend the above analysis to account for finding the attribution of transcoder features through the OV circuit of an attention head.

As before, we want to understand what causes feature i' in the layer l' transcoder to activate on token t. Given attention head h at layer l with $l < l'$, the same arguments as before imply that the contribution of this attention head to feature i' is given by $\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')} \cdot \mathrm{attn}^{(l,h)}\left(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})}; \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{1}:\mathbf{t})}\right)$. This can further be decomposed as

$$\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')} \cdot \left(\sum_{\text{source token } s} \mathrm{score}^{(l,h)}\left(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})}, \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}\right)\mathbf{W}_{\mathbf{OV}}^{(\mathbf{l},\mathbf{h})}\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}\right) \tag{19}$$

$$= \sum_{\text{source token } s} \mathrm{score}^{(l,h)}\left(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})}, \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}\right)\left(\left(\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')}\right)^T \mathbf{W}_{\mathbf{OV}}^{(\mathbf{l},\mathbf{h})}\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}\right) \tag{20}$$

$$= \sum_{\text{source token } s} \mathrm{score}^{(l,h)}\left(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})}, \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}\right)\left(\left(\left(\mathbf{W}_{\mathbf{OV}}^{(\mathbf{l},\mathbf{h})}\right)^T \mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')}\right) \cdot \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}\right). \tag{21}$$

From this, we now have that the contribution of token s at layer l through head h is given by

$$\mathrm{score}^{(l,h)}\left(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})}, \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}\right)\left(\left(\left(\mathbf{W}_{\mathbf{OV}}^{(\mathbf{l},\mathbf{h})}\right)^T \mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')}\right) \cdot \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}\right). \tag{22}$$

The next step is to note that $\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}$ can, in turn, be decomposed into the output of MLP sublayers (or alternatively, transcoder features), the output of attention heads, and the original token embedding. These previous-layer components affect the contribution to the original feature through both the QK circuit of attention and the OV circuit. This means that these previous-layer components can have very nonlinear effects on the contribution. We address this by following the standard practice